Yellowstone RAG Assistant

Project structure and implemented architecture

Reading guide | 23 October 2026

The application answers independent questions about Yellowstone using a small collection of official National Park Service pages. It has two separate flows: ingestion prepares the knowledge base, and the Streamlit application retrieves evidence and generates answers from that stored knowledge.

Project structure

```
Week2/
  app.py                    Streamlit interface
  config/sources.json       Explicit NPS page allowlist
  src/park_rag/
    config.py               Settings and credential validation
    corpus.py               Collection, cleaning, splitting, IDs
    ingest.py               Offline ingestion command
    store.py                Embedding and Pinecone operations
    qa.py                   Retrieval and grounded answers
  data/                     Generated local data, ignored by Git
    preview.json            Collected chunks for inspection
    active.json             Published corpus pointer, once ingested
  tests/                    Automated offline tests
  docs/verification.md      Recorded verification results
  .env.example              Configuration template
  .gitignore                Credentials and generated-file exclusions
  pyproject.toml            Dependencies and tool configuration
  uv.lock                   Exact dependency versions
  README.md                 Setup and run instructions
```

Technology roles

Python connects the components. LangChain provides document objects, text splitting, embedding adapters, prompts and structured generation. Pinecone stores dense vectors and passage metadata. Streamlit displays questions, answers, evidence and collection dates. uv manages the environment and dependencies.

Ingestion architecture

Ingestion runs separately from the question-answering application. It downloads only the configured Yellowstone pages; it does not crawl linked pages or collect content while a user asks a question.

```
sources.json -> Download -> Clean -> Split -> Embed
             -> Pinecone -> Verify records -> active.json
```

Module	Responsibility
config.py	Loads .env and environment settings for credentials, models, index and chunking.
corpus.py	Validates NPS URLs, checks robots.txt, downloads pages, cleans HTML and creates chunks with metadata.
store.py	Creates embedding vectors, connects to or creates the Pinecone index, upserts chunks and fetches records.
ingest.py	Coordinates ingestion and publishes the completed corpus snapshot.

Collection and chunking

The initial allowlist contains Yellowstone safety, pets and hiking pages. Cleaning removes navigation, scripts, footer content and other unwanted HTML. The initial chunk settings are 1,800 characters with a target overlap of 250 characters, preferring text boundaries. These sizes are characters, not tokens.

Each chunk preserves the page title, source URL, document ID, chunk ID and UTC collection timestamp. The page title is also included in the text sent for embedding.

Stable IDs and corpus publication

A document ID is a hash of its canonical URL. A chunk ID combines that document ID with its ordinal position. The namespace identifies a snapshot using the complete chunk content and embedding configuration. Re-ingesting unchanged content overwrites the same IDs in the same namespace.

Changed content creates a new namespace, keeping obsolete chunks out of current retrieval. After the records are visible in Pinecone, ingestion atomically replaces data/active.json. Previous namespaces remain available as retained history; this version does not automatically delete them.

Commands

```
uv run python -m park_rag.ingest --preview
uv run python -m park_rag.ingest
```

Preview downloads and splits the public pages without model or Pinecone calls. Full ingestion needs credentials configured locally.

Question answering architecture

The Streamlit app reads the published corpus configuration and answers each question independently. Previous conversation turns are not passed to the model.

```
Question -> Embed question -> Pinecone retrieval
         -> Evidence and prompt -> Chat model
         -> Citation validation -> Answer or fallback
```

Request flow in qa.py

1. Read data/active.json and check that the selected index and embedding configuration match the published corpus.

2. Embed the question using the same embedding model and dimensions used during ingestion.

3. Search the active Pinecone namespace with a Yellowstone metadata filter, retrieving up to five passages by default.

4. Supply the question and retrieved passages to a LangChain prompt. The chat model is instructed to use only that evidence and to treat source content as data.

5. Request structured output containing supported claims and citation IDs, a partial answer with a coverage gap, or an unsupported result.

6. Validate that the citation IDs exist in the supplied evidence. Missing or unknown citations cause a fallback.

What the user sees

app.py displays the answer status, answer text, supporting NPS links and expandable source excerpts. The sidebar shows the configured source pages and their UTC collection times. The interface states that stored guidance does not verify live park conditions.

Evidence boundaries

The assistant should say when the available documents do not support an answer. For partial evidence, it answers the supported portion and identifies missing coverage. Live closures, current availability and today's conditions cannot be confirmed from the stored corpus.

Code verifies that citation IDs resolve to retrieved passages. Whether a passage actually supports a generated claim remains model-based and requires evaluation; valid citation IDs alone do not prove faithfulness.

Run the application

```
uv run streamlit run app.py
```

Data storage and current status

Location	Stored information
.env	Local credentials and configuration; excluded from Git.
config/sources.json	The explicit list of approved Yellowstone NPS URLs.
Pinecone	Embedding vectors, passage text and source metadata, organized by corpus namespace.
data/preview.json	A local inspection copy of collected chunks and metadata.
data/active.json	The active index, namespace, embedding configuration and source collection details.

Implemented scope

The current implementation uses dense vector retrieval and single-turn questions. It includes collection, cleaning, chunking, embedding and Pinecone adapters, evidence-grounded generation, citation checks, fallback behavior and a Streamlit interface.

LangGraph, live search, persistent conversation or task state, and human approval have not been added. The module boundaries allow later work to wrap retrieval and answer functions in workflows while keeping live tool outputs separate from dated document evidence.

Verification completed

The recorded public-page preview collected three pages and produced 18 chunks. Titles, source URLs, collection timestamps and footer removal were checked. The local suite passed 21 tests; lint and locked uv dependency synchronization also passed. Package APIs and model constructors were checked without credentialed service calls.

Verification still needed

Real embedding generation, Pinecone index creation and upserts, live retrieval calls and generated-answer quality still require a credentialed end-to-end run. Offline mocked tests do not prove service connectivity, faithfulness or latency. data/active.json is created only after successful full ingestion.

Next steps

Copy .env.example to .env and configure keys locally. Run full ingestion, start Streamlit, and check supported questions, unsupported questions, source links and collection dates. Then evaluate answer support and retrieval failures against a fixed question set.

```
uv sync --locked
uv run python -m park_rag.ingest
uv run streamlit run app.py
uv run pytest -q
uv run ruff check .
```

For detailed setup, read README.md. For the recorded checks and remaining gaps, read docs/verification.md.